The `Item` holds the entity disassembled state, as well as the entity version and a timestamp. The version and the timestamp are used for concurrency control as follows:

- An `Item` is readable only from a `Session` that has been started after the cache entry creation timestamp.
- An `Item` entry can be written only if the incoming version is greater than the current one held in the cache entry.

When an entity or a collection is either updated or deleted, Hibernate replaces the cached `Item` entry with a `Lock`, whose concurrency control mechanism works as follows:

- Since it overwrites an `Item` cache entry, the `Lock` object instructs a concurrent `Session` to read the entity or the collection from the database.
- If at least one `Session` has managed to lock this entry, any write operation is forbidden.
- A `Lock` entry is writable only if the incoming entity state has a version which is newer than the one contained in the `Lock` object, or if the current `Session` creation timestamp is greater than the `Lock` timeout threshold.

If the database transaction is rolled back, the current cache entry holds a `Lock` instance which cannot be undone to the previous `Item` state. For this reason, the `Lock` must time out to allow the cache entry to be replaced by an actual `Item` cache entry.

For Ehcache, the default `Lock` timeout is 120 seconds, and it can be customized via the `net.sf.ehcache.hibernate.cache_lock_timeout` configuration property.

The `READ_WRITE` concurrency strategy offers a write-through caching mechanism without requiring JTA transactions.

However, for heavy write contention scenarios, when there is a chance of rolling back transactions, the soft locking concurrency control can lead to having other concurrent transactions hitting the database for the whole duration of the lock timeout period. For this kind of situations, the `TRANSACTIONAL` concurrency strategy might be more suitable.

15.5.6.4 TRANSACTIONAL

While `READ_WRITE` is an asynchronous write-through cache concurrency strategy, `TRANSACTIONAL` uses a synchronous caching mechanism.

To enlist two data sources (the database and the second-level cache) in the same global transaction, a JTA transaction manager is needed. When using Java EE, the application server provides JTA transactions by default. For stand-alone enterprise applications, there are multiple transaction managers to choose from (e.g. Bitronix, Atomikos, Narayana).

For JTA transactions, Ehcache offers two failure recovery options: `xa_strict` and `xa`.

15.5.6.4.1 XA_Strict mode

In this mode, the second-level cache exposes a `XAResource` interface so that it can participate in the two-phase commit (2PC) protocol.

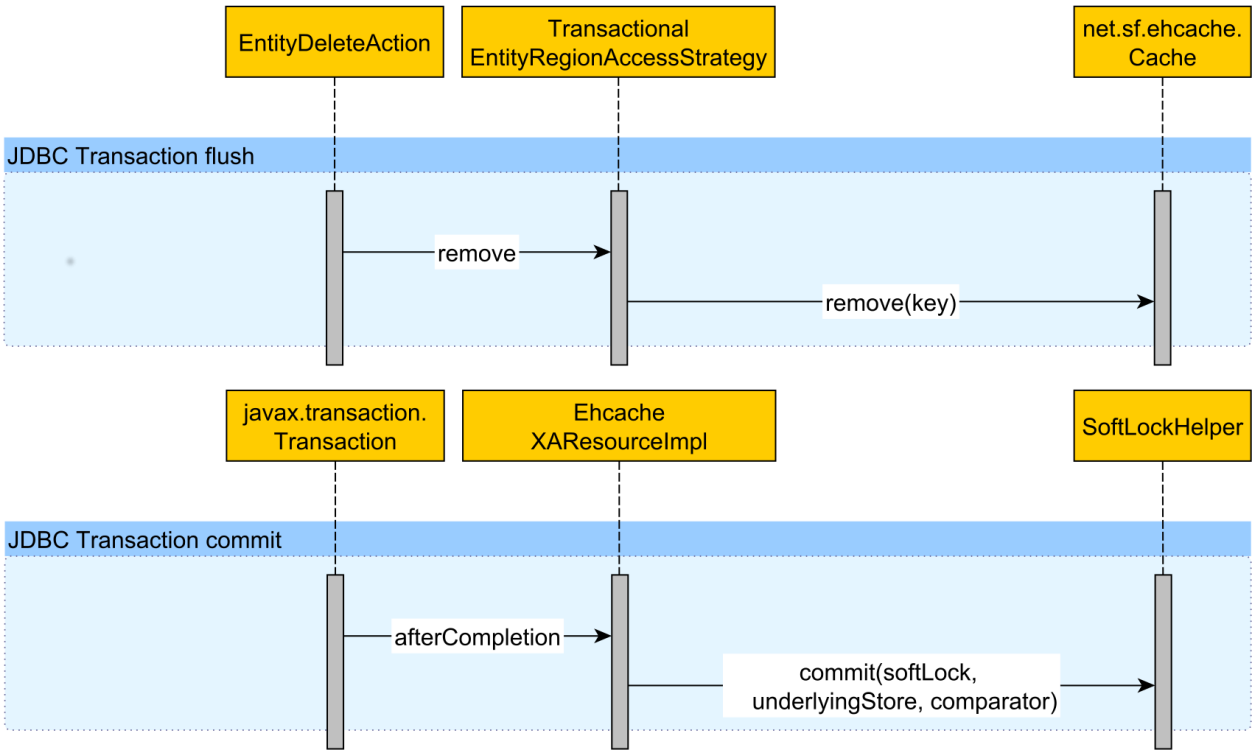

Figure 15.14: TRANSACTIONAL XA_Strict flow

The entity state is modified both in the database and in the cache, but these changes are isolated from other concurrent transactions, and they become visible once the current XA transaction gets committed.

The database and the cache remain consistent even in the case of an application crash.

15.5.6.4.2 XA mode

If only one `DataSource` participates in a global transaction, the transaction manager can apply the one-phase commit optimization. The second-level cache is managed through a `javax.transaction.Synchronization` transaction callback. The `Synchronization` does not actively participate in deciding the transaction outcome, therefore following the current database transaction outcome:

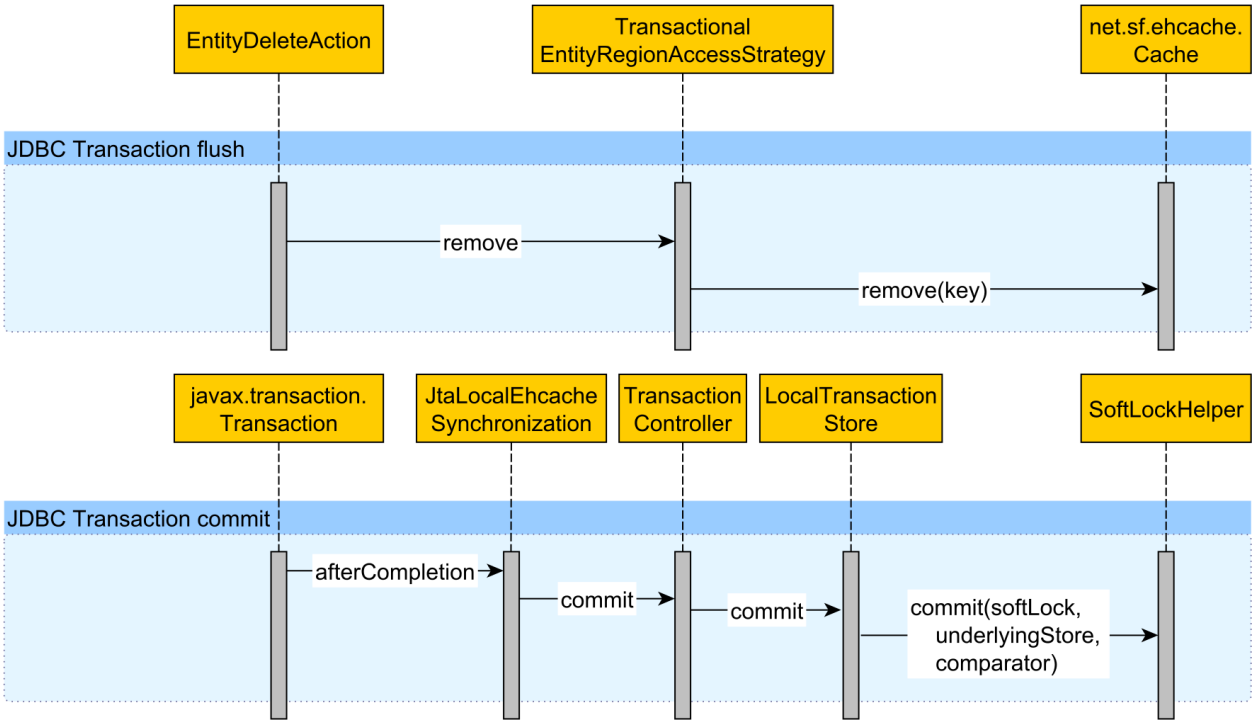

Figure 15.15: TRANSACTIONAL XA flow

This mode trades durability for a lower response time, and in the case of a server crash (happening in between the database transaction commit and the second-level cache transaction callback), the two data sources will drift apart. This issue can be mitigated if entities employ an optimistic concurrency control mechanism, so, even if the application reads stale data, it will not lose updates upon writing it back.

The following examples are going to reuse the same entities that were previously employed, and the only thing that differs is that the `Post` and `PostComment` entities, as well as the `comments` collections, are using the `@Cache(usage = CacheConcurrencyStrategy.TRANSACTIONAL)` annotation. The transaction boundaries are managed by Spring framework, and the actual JTA transaction logic is coordinated by Bitronix Transaction Manager.

15.5.6.4.3 Inserting TRANSACTIONAL cache entries

When running the same example used for inserting NONSTRICT_READ_WRITE cache entries, Hibernate generates the following output:

```
INSERT INTO post (title, version, id)
VALUES ('High-Performance Java Persistence', 0, 1)

INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'JDBC part review', 1)
INSERT INTO post_comment (post_id, review, id)
VALUES (1, 'Hibernate part review', 2)

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=00],
Entries: {}

--Load Post entity and comments collection

SELECT pc.post_id AS post_id3_1_0_, pc.id AS id1_1_0_, pc.review AS review2_1_1_
FROM   post_comment pc
WHERE  pc.post_id = 1

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}
```

Just like READ_WRITE, the TRANSACTIONAL cache concurrency strategy is write-through for entities (unless using the identity generator in which case it is read-through), and read-though for collections.

15.5.6.4.4 Updating TRANSACTIONAL cache entries

Because the `TRANSACTIONAL` cache is synchronous, all changes are applied directly to cache, as illustrated by the following example:

```
doInJPA(entityManager -> {
    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());

    Post post = entityManager.find(Post.class, 1L);
    post.setTitle("High-Performance Hibernate");
    PostComment comment = post.getComments().remove(0);
    comment.setPost(null);

    entityManager.flush();

    printCacheRegionStatistics(Post.class.getName());
    printCacheRegionStatistics(Post.class.getName() + ".comments");
    printCacheRegionStatistics(PostComment.class.getName());

    LOGGER.debug("Commit after flush");
});
printCacheRegionStatistics(Post.class.getName());
printCacheRegionStatistics(Post.class.getName() + ".comments");
printCacheRegionStatistics(PostComment.class.getName());
```

For which Hibernate generates the following output:

```
Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post,
Statistics: SecondLevelCacheStatistics[hitCount=1,missCount=0,putCount=1],
Entries: {1=CacheEntry(Post)[1,High-Performance Java Persistence,0]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.Post.comments,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=1,putCount=1],
Entries: {1=CollectionCacheEntry[1,2]}

Region: com.vladmihalcea.book.hpjp.hibernate.cache.PostComment,
Statistics: SecondLevelCacheStatistics[hitCount=0,missCount=0,putCount=2],
Entries: {1=CacheEntry(PostComment)[1,JDBC part review],
          2=CacheEntry(PostComment)[1,Hibernate part review]}
```